NEPALENGINEERING COLLEGE

(Affiliated to Pokhara University)

Changunarayan – 04, Bhaktapur, Nepal

Individual Course Project Report on

Handwritten Digit Recognizer

Submitted to

Department of Computer Science and Engineering

Submitted by

Abhinav Khatiwada [023-303]

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Department of Computer Engineering, Nepal Engineering College, for providing me with the opportunity to complete this project as part of the Image Processing and Pattern Recognition course. I am especially thankful to my course instructor, **JUNIOR PROFESSOR HARIMOHAN KHATRI**, for his valuable guidance, suggestions, and support throughout the development of the project, **"HANDWRITTEN DIGIT RECOGNITION USING HOG FEATURE EXTRACTION AND SVM CLASSIFICATION."**I would also like to thank the faculty members and my colleagues for their support and feedback during the project. Finally, I am grateful to everyone who contributed directly or indirectly to the successful completion of this project.

ABSTRACT

Handwritten digit recognition is an important application of image processing and pattern recognition, with uses in areas such as document processing, form recognition, and automated data entry. This project presents a handwritten digit recognition system that identifies digits from 0 to 9 using the MNIST dataset. The system uses image preprocessing to convert input images into a suitable 28×28 format and applies Histogram of Oriented Gradients (HOG) for feature extraction. The extracted features are then classified using a Support Vector Machine (SVM) with an RBF kernel.The trained model achieved a test accuracy of **97.5%** on the MNIST test set. In addition to dataset-based prediction, the system provides a live drawing interface that allows users to draw a handwritten digit and receive the predicted result. Image preprocessing also includes centering the digit based on its center of mass to improve consistency with the MNIST data. The project demonstrates that combining HOG feature extraction with SVM classification can provide an effective and lightweight approach for handwritten digit recognition.

Table of Contents

List of figures

CHAPTER-1: INTRODUCTION

1.1 Background

Handwritten digit recognition is the process of automatically identifying numerical digits from handwritten images. It is an important application of image processing and pattern recognition, where a computer analyzes the visual characteristics of an image and determines the corresponding digit. In this project, handwritten digits from 0 to 9 are recognized using the MNIST dataset, HOG feature extraction, and an SVM classifier.Image processing involves manipulating and analyzing digital images to improve their quality and prepare them for further analysis. In the proposed system, image preprocessing is used to convert the input digit into a suitable format, including resizing, cropping, thresholding, and centering before feature extraction.Pattern recognition focuses on identifying meaningful patterns or characteristics within data and assigning them to predefined classes. For handwritten digit recognition, the visual patterns of each digit are extracted and used to distinguish one digit from another. The proposed system uses Histogram of Oriented Gradients (HOG) to represent the shape and edge information of the digit and Support Vector Machine (SVM) to perform classification.Automatic digit recognition is useful because it can reduce manual data entry and improve the speed and efficiency of processing handwritten information. It can be applied in areas such as digitizing documents, recognizing handwritten forms, postal processing, and educational applications. Therefore, developing an accurate and efficient digit recognition system provides a practical application of image processing and pattern recognition techniques.

1.2 Problem Statement

Recognizing handwritten digits automatically is challenging because people write the same digit in many different ways. Variations in writing style, stroke thickness, size, position, and orientation can significantly change the appearance of a digit. For example, one person may write a digit with thin strokes while another may use thicker strokes or a different shape.Another challenge is that some digits have similar visual characteristics. Digits such as 3 and 5 or 4 and 9 can sometimes appear similar depending on the handwriting style. In addition, digits may be shifted, rotated slightly, or positioned differently within an image, making direct classification more difficult.Therefore, a suitable image preprocessing and feature extraction method is required to represent the important characteristics of handwritten digits. This project addresses the problem by preprocessing the input image, extracting Histogram of Oriented Gradients (HOG) features to capture shape and edge information, and using an SVM with an RBF kernel to classify the digit. The system is trained and evaluated using the MNIST dataset and also provides a live drawing interface for handwritten digit prediction.

1.3 Objectives

- To develop a system for recognizing handwritten digits from 0 to 9.
- To preprocess handwritten digit images into a suitable 28×28 format for recognition.
- To extract useful shape and edge features using Histogram of Oriented Gradients (HOG).
- To classify handwritten digits using a Support Vector Machine (SVM) with an RBF kernel.
- To implement a live drawing interface for real-time handwritten digit prediction.

1.4 Scope

The scope of this project is focused on recognizing handwritten digits from 0 to 9 using the MNIST dataset. The system uses image preprocessing, HOG feature extraction, and an SVM classifier with an RBF kernel. Input images are converted into a **28×28** format before feature extraction and classification.The project also includes a live drawing interface where users can draw a digit and obtain its predicted class. In addition, the system provides functionality for testing the trained model on custom handwritten digit images. The project mainly focuses on single-digit recognition and does not cover handwritten words, letters, or multi-digit sequences.

1.5 Limitations

Although the proposed system provides good performance on the MNIST test data, it has some limitations. The prediction performance can be affected by the quality, size, position, and writing style of the input image.The SVM model was trained using 5,000 training samples from the MNIST dataset rather than the complete training set, which may limit its ability to handle a wider range of handwriting styles. The reported 97.5% accuracy is based on the MNIST test set, while extensive real-world testing using personal handwritten samples has not yet been completed.Another limitation is that the current SVM model was trained without probability estimation. Therefore, the system provides the predicted digit but does not provide a calibrated probability or confidence score for the prediction.

CHAPTER-2: LITERATURE REVIEW

2.1 Handwritten Digit Recognition

Handwritten digit recognition is a pattern recognition problem in which a computer identifies numerical digits from images. Earlier approaches generally relied on manually designed image features followed by a classification algorithm. Features based on pixel intensity, edges, contours, and geometric properties were commonly used to represent handwritten characters before classification.Machine learning methods have provided more effective approaches for handwritten digit recognition. Support Vector Machines (SVM), neural networks, and other classifiers have been widely used to learn patterns from labeled digit images. In particular, the MNIST dataset has become a standard benchmark for evaluating handwritten digit recognition systems. Research on handwritten digit recognition has also demonstrated the effectiveness of neural-network-based methods for learning features directly from images [1].The choice of feature representation has an important effect on recognition performance. Instead of using raw pixel values directly, feature extraction methods can represent the structural characteristics of a digit in a more meaningful form. In this project, HOG is used to capture the edge and shape information of handwritten digits, followed by SVM classification.

2.2 MNIST Dataset

The Modified National Institute of Standards and Technology (MNIST) dataset is one of the most widely used datasets for handwritten digit recognition. It contains 60,000 training images and 10,000 test images representing digits from 0 to 9 [2]. Each image is a grayscale image of size 28×28 pixels.The digits in MNIST are size-normalized and centered within a fixed 28×28 image area. This standardized format makes the dataset suitable for comparing different machine learning and pattern recognition methods [2]. Because of its relatively simple structure and large number of labeled examples, MNIST is commonly used as a benchmark for evaluating handwritten digit recognition algorithms.In this project, MNIST is used as the primary dataset for training and evaluating the HOG-SVM model. Although the complete MNIST dataset contains 60,000 training samples and 10,000 test samples, the implemented model uses a subset of 5,000 training samples for training.

2.3 HOG Feature Extraction

Histogram of Oriented Gradients (HOG) is a feature extraction technique that describes the shape of an object using local gradient information. The method was introduced by Dalal and Triggs and demonstrated the effectiveness of gradient-based orientation histograms for visual recognition tasks [3].HOG first calculates the image gradients**,** which describe changes in pixel intensity. From these gradients, the magnitude and direction of the local edges are obtained. The image is then divided into small cells, and the gradient directions are grouped into orientation bins to form histograms. Neighboring cells are combined into blocks, where normalization is applied to reduce the effect of illumination and contrast variations [3].For handwritten digit recognition, HOG is useful because the shape of a digit is strongly represented by its edges and stroke directions. In this project, HOG converts each preprocessed 28×28 digit image into a numerical feature vector that is then provided to the SVM classifier.

2.4 Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm commonly used for classification. The basic idea of SVM is to find a decision boundary that separates different classes while maximizing the margin between the classes. The data points that have the greatest influence on the decision boundary are called support vectors [4].For data that cannot be separated effectively using a simple linear boundary, SVM can use the kernel trick to map the input features into a higher-dimensional feature space. This allows nonlinear relationships between classes to be modeled without explicitly calculating the higher-dimensional representation.This project uses the Radial Basis Function (RBF) kernel, which is suitable for modeling nonlinear relationships between HOG feature vectors. The RBF kernel allows the SVM to create flexible decision boundaries for distinguishing between different handwritten digits. SVMs have been established as effective learning methods for classification and pattern recognition [4].

Approach	Feature Extraction	Classifier	Main Advantage	Main Limitation
Raw Pixels + SVM	None	SVM	Simple implementation	Sensitive to image variations
HOG + SVM	HOG	RBF-SVM	Captures shape and edge information	Requires manual feature extraction
CNN	Learned automatically	Neural Network	Learns complex features directly	More computationally demanding

Table 1: Comparison of Handwritten Digit Recognition Approaches

For this project, **HOG + SVM** was selected because it provides a good balance between recognition performance, computational complexity, and implementation simplicity. HOG is well suited for representing the shape and stroke information of handwritten digits, while the RBF-SVM can classify the extracted features using nonlinear decision boundaries. This approach is also appropriate for a small-scale academic project where an efficient and understandable machine learning pipeline is preferred.

CHAPTER-3: SYSTEM ANALYSIS AND DESIGN

3.1 System Requirements

The system requirements describe the main functions and qualities required for the handwritten digit recognition system. The proposed system consists of image input, preprocessing, feature extraction, classification, and result display components.

3.2 Functional Requirement

- **Digit Input:** The system should allow the user to draw a handwritten digit through the live drawing interface and support testing with custom handwritten digit images.

- **Image Preprocessing:** The input image should be converted into a suitable format through grayscale conversion, thresholding, contour detection, cropping, resizing, and centering.

- **HOG Feature Extraction:** The system should extract Histogram of Oriented Gradients (HOG) features from the preprocessed image.

- **Digit Classification:** The extracted HOG features should be provided to the trained SVM classifier to predict a digit from 0 to 9.

- **Result Display:** The predicted digit should be displayed to the user through the application interface.

- **Custom Sample Testing:** The system should allow custom handwritten images to be processed and evaluated using the trained model.

3.3 Non-Functional Requirements

- **Accuracy:** The system should provide reliable digit classification performance.

- **Speed:** Prediction should be performed quickly after the input is provided.

- **Reliability:** The system should produce consistent results for properly formatted input images.

- **Usability:** The live drawing interface should be simple and easy to operate.

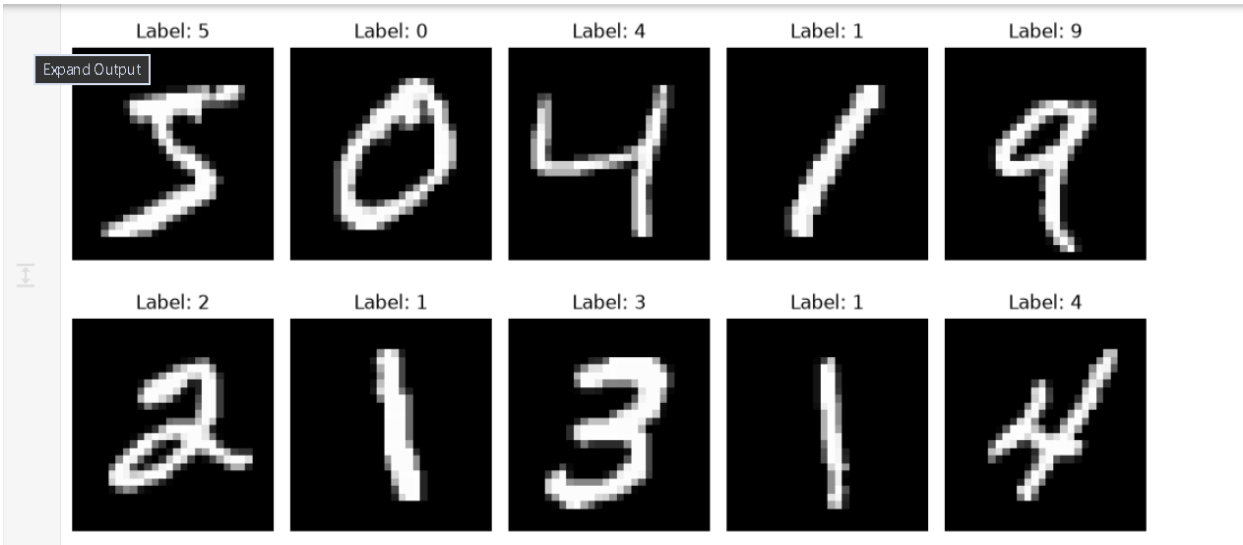


Figure 1.Sample images from the MNIST dataset

3.4 System Architecture

The proposed system follows a sequential image processing and classification pipeline. The overall architecture is:

User Input → Image Preprocessing → HOG Feature Extraction → SVM Classifier → Predicted Digit → Display Result

The user input can be a digit drawn using the live drawing interface or a custom handwritten image. The input is first passed through the preprocessing stage, where the image is converted and normalized into a suitable format.After preprocessing, HOG is applied to extract features representing the edge directions and shape characteristics of the digit. These features are then passed to the trained SVM classifier, which predicts the corresponding digit class. Finally, the predicted result is displayed to the user.

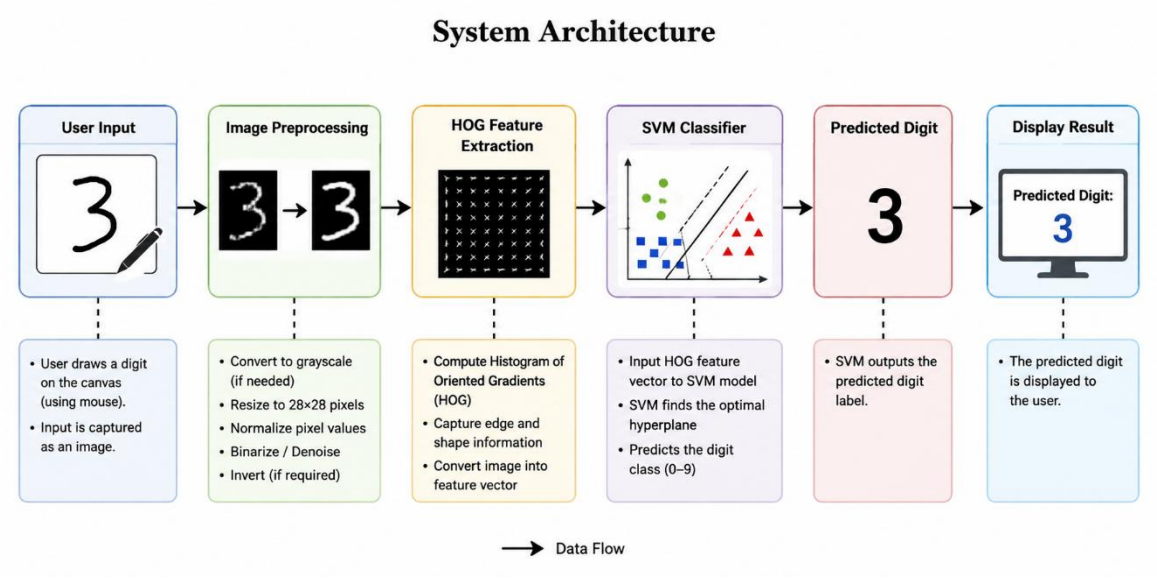


Figure 2.System Architecture of the Handwritten Digit Recognition System

3.5 system flowchart

The system flowchart illustrates the sequence of steps followed by the handwritten digit recognition system from input to final prediction. The process begins when the user provides a handwritten digit through the drawing interface. The input image is first converted to grayscale and thresholded to separate the digit from the background. The system then detects the contour of the digit, crops the relevant region, resizes it, and centers it within a 28×28 image. The processed image is then passed to the HOG feature extraction stage, where shape and edge information is converted into numerical features. These features are given to the trained SVM classifier, which predicts the corresponding digit from 0 to 9. Finally, the predicted digit is displayed to the user, completing the recognition process.

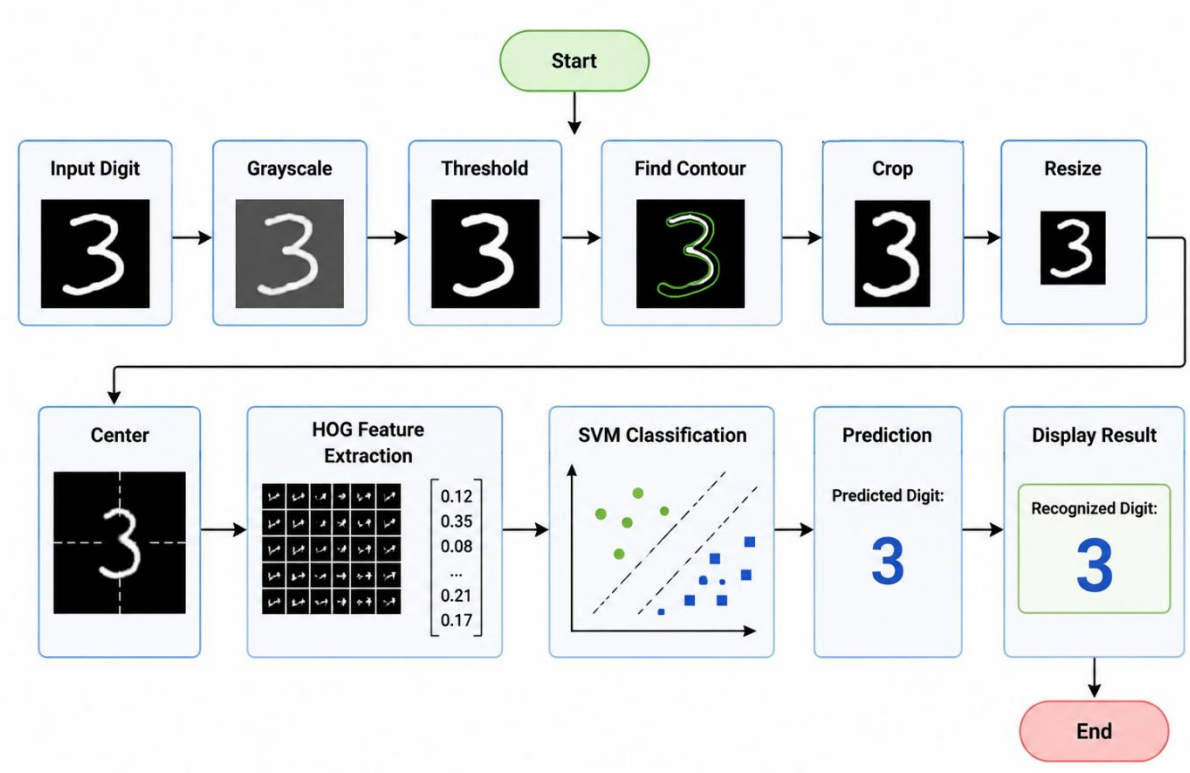


Figure 3.System Flowchart of the Handwritten Digit Recognition System

CHAPTER-4: IMPLEMENTATION

4.1 Implementation

The handwritten digit recognition system was implemented using Python and a set of image processing and machine learning libraries. The implementation consists of several stages, including dataset preparation, image preprocessing, HOG feature extraction, SVM training and classification, and development of a live drawing interface. The system was trained using the MNIST handwritten digit dataset and uses an SVM classifier with an RBF kernel to recognize digits from 0 to 9. The implemented system also allows users to draw digits through an OpenCV-based interface and obtain a predicted result.

4.2 Development Environment and Tools

The system was developed using Python in a Jupyter Notebook environment. Python was selected because it provides suitable libraries for image processing, feature extraction, machine learning, and visualization. OpenCV was used for image processing and development of the live drawing interface. NumPy was used for numerical and array operations, while Scikit-image was used for HOG feature extraction. Scikit-learn was used to load the MNIST dataset and implement the Support Vector Machine classifier. Matplotlib was used for displaying and visualizing digit images, and Joblib was used to save and load the trained SVM model.

The main development tools and libraries used in the project are summarized below.

Tools/Library	Purpose
Python(3.12)	Main programming language
Jupyter Notebook	Model development and experimentation
OpenCV	Image processing and live drawing interface
NumPy	Numerical and array operations
Scikit-image	HOG feature extraction
Scikit-learn	Dataset handling and SVM classification
Matplotlib	Image visualization
Joblib	Saving and loading the trained model

Table 4.1: Development Environment

4.3 Dataset Loading and Preparation

The MNIST handwritten digit dataset was used for training and evaluating the proposed recognition system. The dataset contains grayscale images of handwritten digits from 0 to 9, where each image has a resolution of 28×28 pixels. The dataset provides a standardized format and a variety of handwriting styles, making it suitable for evaluating handwritten digit recognition algorithms.The dataset was loaded using the `fetch_openml()` function from Scikit-learn. The image data was initially represented as flattened vectors containing 784 pixel values, corresponding to the 28×28 image dimensions. The labels were converted into integer values representing the digits from 0 to 9.For the implemented HOG-SVM model, a subset of 5,000 training samples was used to reduce training time and computational requirements. The images were then processed and converted into HOG feature vectors before being supplied to the SVM classifier.

```
# Load MNIST dataset
print("Loading MNIST dataset...")

mnist = fetch_openml('mnist_784', version=1, as_frame=False)

X = mnist.data
y = mnist.target.astype(int)

print("Dataset loaded successfully!")
print("Image data shape:", X.shape)
print("Labels shape:", y.shape)
```

```
Loading MNIST dataset...
Dataset loaded successfully!
Image data shape: (70000, 784)
Labels shape: (70000,)
```

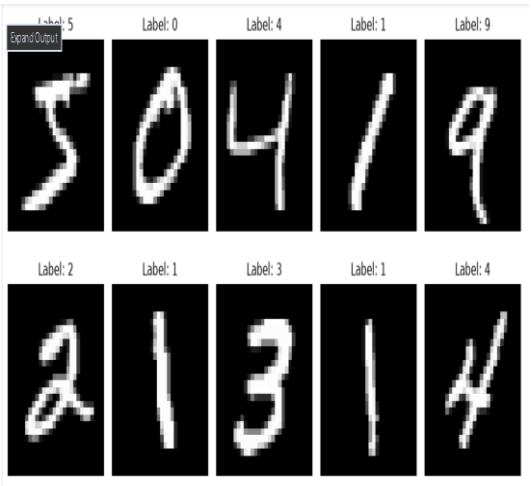


Figure 4.: MNIST Dataset Loading

4.4 Image Preprocessing

Image preprocessing is performed to convert the user-provided handwritten digit into a format that is more consistent with the MNIST images used during model training. The preprocessing stage reduces variations in image size, position, and background before feature extraction. First, the input image is converted into grayscale to simplify the image representation. Thresholding is then applied to separate the handwritten digit from the background. The system detects the contour of the digit and determines its bounding region. The detected region is cropped to remove unnecessary background areas.The cropped digit is then resized while maintaining its shape and placed into a 28×28 image canvas. Finally, the digit is centered using its center of mass so that its position is more consistent with the centered digits in the MNIST dataset. The resulting 28×28 image is then passed to the HOG feature extraction stage.

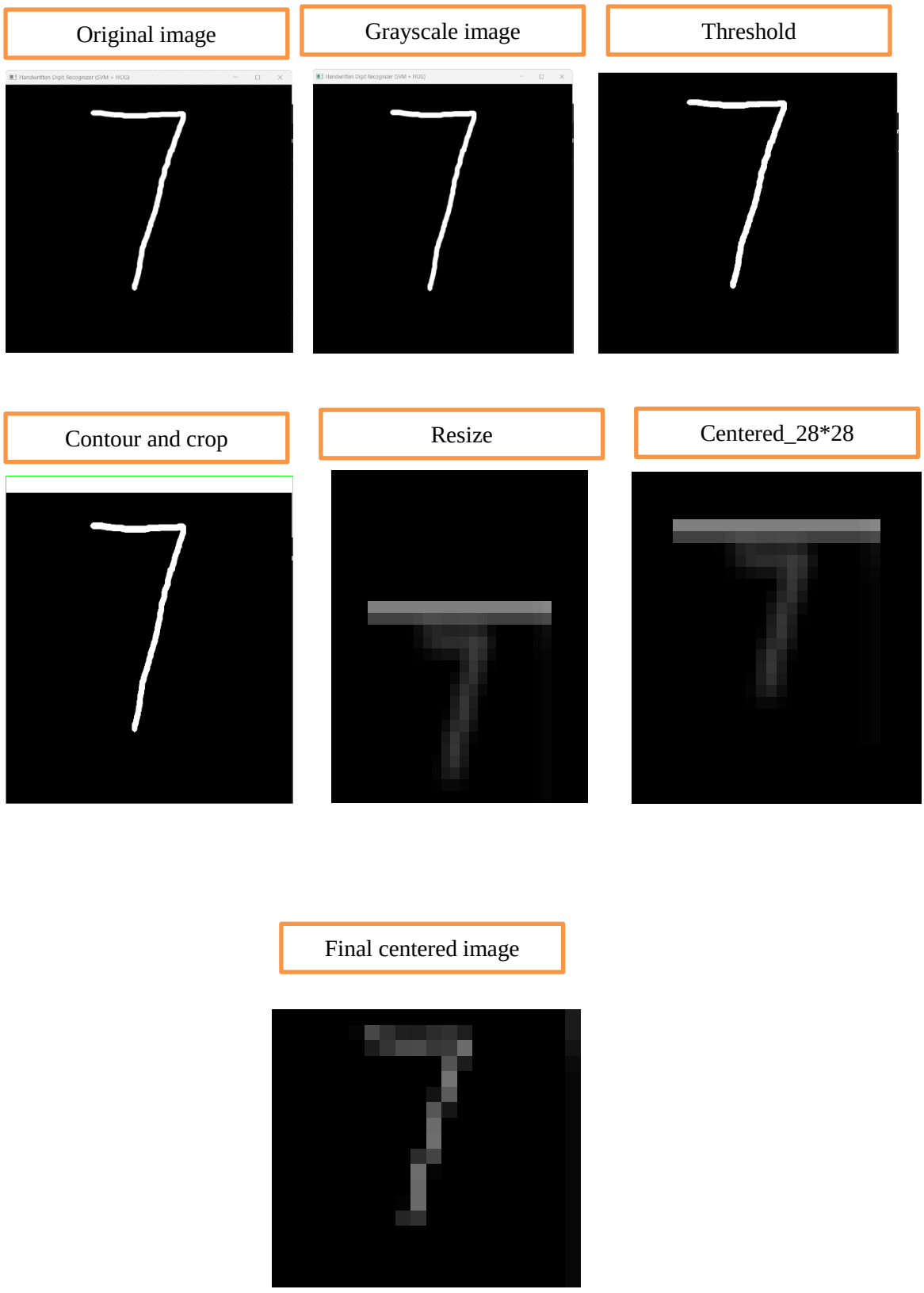


Figure 4.3: Image Preprocessing Stages for a Handwritten Digit

4.5 HOG Feature Extraction

Histogram of Oriented Gradients (HOG) is used to extract shape and edge information from the preprocessed digit image. Since handwritten digits are mainly characterized by their stroke directions and shapes, HOG provides a suitable representation for the SVM classifier.The HOG descriptor calculates local image gradients and determines the magnitude and direction of edges. The image is divided into small cells, and gradient directions are grouped into orientation bins. Neighboring cells are combined into blocks and normalized to produce a more robust feature representation.In this project, the HOG descriptor uses 9 orientation bins, a 4×4 pixel cell size, and a 2×2 cell block size with L2-Hys normalization. The resulting descriptor produces a 1296-dimensional feature vector for each 28×28 input image. These feature vectors are used as the input to the SVM classifier.

```
# Extract HOG features from training images

hog_train = []

for image in X_train_images:

    features = hog(
        image,
        orientations=9,
        pixels_per_cell=(4, 4),
        cells_per_block=(2, 2),
        block_norm='L2-Hys'
    )

    hog_train.append(features)

hog_train = np.array(hog_train)

print("Training HOG feature shape:", hog_train.shape)
```

```
Training HOG feature shape: (5000, 1296)
```

Figure 5.HOG Feature Vector Extraction

4.6 SVM Training and Classification

The extracted HOG feature vectors are used as input to a Support Vector Machine (SVM) classifier. SVM is a supervised machine learning algorithm that determines decision boundaries between different classes. Since handwritten digit classes may not be linearly separable in the HOG feature space, an RBF kernel is used to model nonlinear relationships between the extracted features.The SVM classifier was trained using 5,000 MNIST training samples represented by their HOG feature vectors. The trained model learns the relationship between the HOG features and their corresponding digit labels from 0 to 9.After training, the SVM model was saved using Joblib so that it could be loaded by the live prediction application without retraining the classifier. During prediction, a processed input image is converted into its HOG feature vector and passed to the trained SVM model. The classifier then returns the predicted digit.The trained HOG-SVM model achieved an accuracy of 97.45% on the test data used for evaluation.

```
# Train the SVM classifier

print("Training SVM...")

svm = SVC(
    kernel='rbf',
    gamma='scale'
)

svm.fit(hog_train, y_train_small)

print("Training completed!")
```

```
Training SVM...
Training completed!
```

```
# Predict on the testing data

print("Predicting...")

predictions = svm.predict(hog_test)

accuracy = accuracy_score(y_test, predictions)

print(f"Accuracy: {accuracy * 100:.2f}%")
```

```
Predicting...
Accuracy: 97.45%
```

Figure 6.HOG Feature Vector Extraction

4.7 Application and Custom Image Testing

A live drawing application was developed using OpenCV to allow users to provide handwritten digits directly through a computer mouse. The application provides a black drawing canvas on which the user can draw a single digit. When the prediction command is triggered, the drawn image is passed through the same preprocessing and HOG feature extraction stages used by the recognition system.The extracted HOG features are then supplied to the trained SVM classifier, which predicts the corresponding digit. The predicted result is displayed through the application interface. The application also provides basic controls for clearing the drawing and closing the program. The `C` key is used to clear the canvas, the `P` key is used to perform prediction, and the `ESC` key is used to exit the application.In addition to the live drawing interface, custom handwritten digit images can be processed using the same preprocessing and classification pipeline. This provides a practical way to observe how the trained model performs on inputs outside the standard MNIST samples.

CHAPTER-5: RESULTS AND DISCUSSION

The implemented handwritten digit recognition system was evaluated using the HOG feature extraction technique and an SVM classifier. The evaluation focused on the classification performance of the trained model and its ability to recognize handwritten digits from input images. In addition to testing the model on the dataset, the trained model was integrated with a live drawing application to evaluate its practical operation.

5.1 Model Performance

The HOG-SVM model achieved an accuracy of **97.45%** on the test data. This result indicates that the combination of HOG features and the SVM classifier was effective for handwritten digit classification. HOG provides information about the shape and edge structure of the handwritten digits, while the SVM classifier uses these extracted features to distinguish between the ten digit classes from 0 to 9.

PARAMETRR	RESULTS
Dataset	MNIST
Number of classes	10
Training samples	5000
Feature extraction	HOG
Classifier	SVM
SVM kernel	RBF
HOG feature dimensions	1296
Accuracy	97.45%

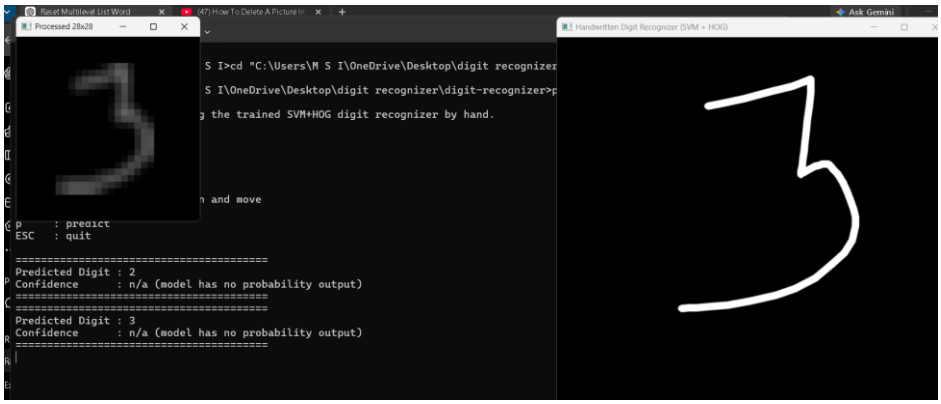
Table 5.1: HOG-SVM Model Performance

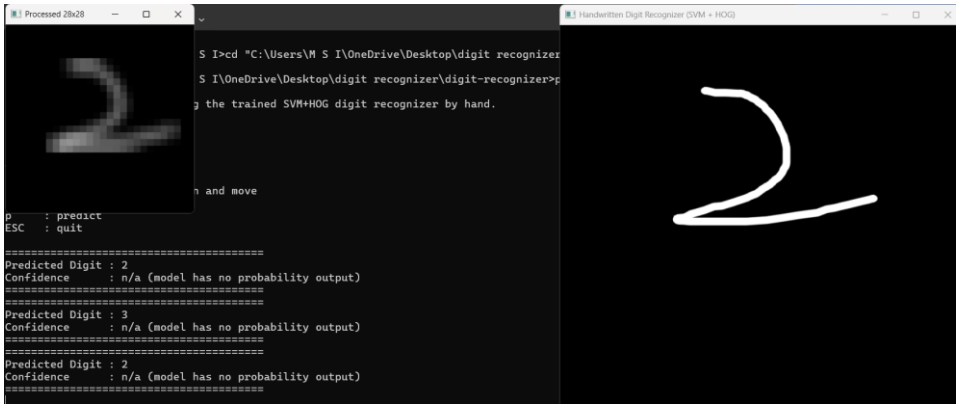
```
# Predict on the testing data

print("Predicting...")

predictions = svm.predict(hog_test)

accuracy = accuracy_score(y_test, predictions)

print(f"Accuracy: {accuracy * 100:.2f}%")
```

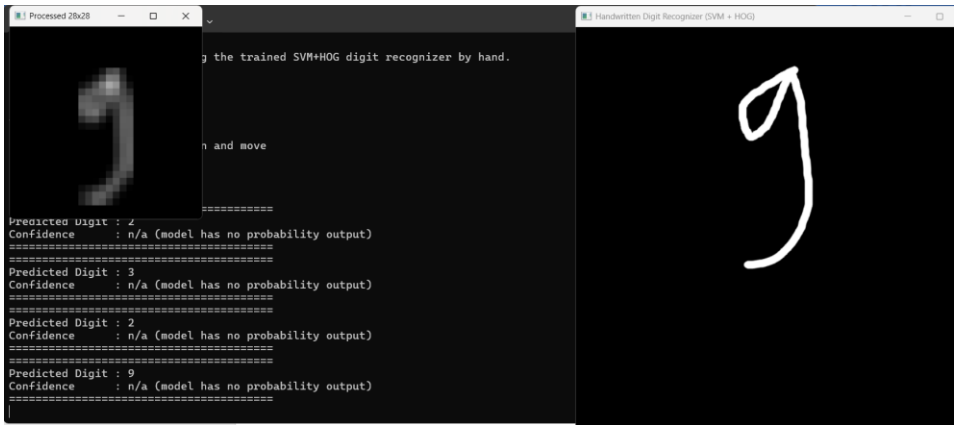
```
Predicting...
Accuracy: 97.45%
```

Figure 7.Accuracy Screenshot

5.2 Prediction Results







5.3 Discussion of Results

The obtained accuracy of 97.45% demonstrates that the HOG-SVM approach provides effective recognition of handwritten digits. HOG represents the structural characteristics of the digits through local gradient information, allowing the classifier to focus on stroke direction and shape rather than relying directly on individual pixel values. The RBF-based SVM is capable of handling the nonlinear separation between different digit classes. However, accuracy on the MNIST test data does not necessarily represent the exact performance of the live drawing application. MNIST images have a standardized 28×28 format, while user-generated drawings may differ in stroke thickness, position, size, and shape. These differences can introduce errors during live prediction even when the model performs well on the dataset.The preprocessing stage helps reduce these differences by thresholding the input, detecting the digit contour, cropping unnecessary background, resizing the digit, and centering it before HOG feature extraction. This allows the live input to be converted into a representation that is more consistent with the data used during model training.